

Intrusion Detection for IoT Traffic in O-RAN Edge Data Centres: Why Accuracy Alone Is Not Enough

First A. Author, Second B. Author, and Third C. Author

Department of Computing and Communications

Institution Name

City, Country

{first.author, second.author, third.author}@institution.edu

Abstract—Edge data centres in Open Radio Access Network (O-RAN) deployments receive telemetry from very large numbers of IoT devices, and compromised IoT devices are a documented source of control-plane signalling overload. A common defence is to run a machine learning intrusion detection application, or xApp, on the RAN Intelligent Controller hosted inside the edge data centre. Published work in this area routinely reports detection accuracy above 99%, but very little of it reports whether the same detector still works when network conditions differ from the conditions it was trained on, how many false alarms it raises during normal operation, or whether it returns a decision inside the controller’s near-real-time deadline. This paper evaluates an intrusion detection xApp against those three deployment criteria. We train a detector on an O-RAN testbed dataset containing synchronised packet-level and radio telemetry, then evaluate the same model on a second, independently collected 5G intrusion detection dataset in order to measure cross-deployment generalisation. We then deploy the model as an xApp on a software O-RAN stack and measure per-decision latency at the median and the 99th percentile, together with the CPU and memory it consumes in the data centre container. Our results show that the best in-distribution model falls from an F_1 score of 99.25% to 71.39% when transferred to an independently collected deployment, a drop of 27.9 points; that at its default operating threshold the same model raises 682 alerts per hour on benign-only traffic and attains an operational precision of only 41.2% once a realistic attack base rate is applied; and that only the gradient-boosted tree ensemble meets a 10 ms 99th-percentile budget, and only up to an offered load of roughly 2,300 decisions per second. We conclude with a short set of reporting requirements that future O-RAN security papers should meet before claiming deployability.

Index Terms—O-RAN, edge data centre, IoT security, intrusion detection, xApp, RAN Intelligent Controller, model generalisation, false positive rate

DRAFT NOTICE — NOT FOR SUBMISSION. Every numerical result in this document is a synthetic placeholder generated to illustrate structure and expected experimental depth. No measurement reported here was actually taken. Replace all values marked % [SYNTHETIC] in the source with real measurements, then set `\synthdraftfalse` in the preamble to remove this banner.

I. INTRODUCTION

Open Radio Access Network (O-RAN) architectures disaggregate the base station into radio, distributed and centralised units connected by open interfaces, and place a RAN Intelligent Controller (RIC) alongside them to host third-party control logic [1]–[3]. In most commercial designs the near-real-time RIC is not deployed at the cell site but inside a

regional edge data centre that aggregates traffic from tens to hundreds of cells [4], [5]. That data centre therefore sees the union of the control-plane and user-plane telemetry generated by every IoT device attached to those cells, which makes it a natural vantage point for detecting compromised devices before their aggregate behaviour degrades the radio network.

The motivating threat is well documented. Large IoT botnets have repeatedly been assembled from devices with weak or default credentials [6], [7], and in cellular deployments the damage is not limited to volumetric flooding of an external target. Bursts of attach, detach and radio resource control transitions from a synchronised population of devices impose a signalling load that scales with the number of state changes rather than with the number of transmitted bytes, and this class of signalling storm has been shown to degrade a cellular network well before any link saturates [8]. Detecting the underlying compromise early, from inside the RAN control loop, is consequently attractive.

The natural implementation is a detection xApp: a containerised application that subscribes to E2 service model reports, extracts features, classifies traffic, and issues control actions through the same interface [9], [10]. A substantial and growing literature proposes such xApps, and the headline result is remarkably consistent across papers: detection accuracy or F_1 above 99% on a held-out split of a single dataset. What is far less consistent is any evidence that the detector would survive contact with a production network.

Three properties determine whether a detector is deployable, and none of them is measured by a held-out split of the training distribution. The first is cross-deployment generalisation. A model trained on telemetry from one radio configuration, device population and traffic mix must classify correctly in a different one, and network intrusion detection models are known to transfer poorly between independently collected datasets [11], [12]. The second is alert burden. A false positive rate that appears negligible on a class-balanced benchmark becomes an unmanageable alert volume once it is multiplied by production flow rates and discounted by a realistic attack base rate, a consequence of the base-rate fallacy identified for intrusion detection more than two decades ago [13], [14]. The third is latency conformance. The near-real-time RIC control loop is specified to operate between 10 ms and 1 s [15]; a detector whose tail latency exceeds the budget under load does

not merely respond slowly, it fails to close the loop at all.

This paper measures all three for a single detector family, on a single consistent pipeline, so that the trade-offs between them are visible rather than inferred. Our contributions are as follows.

- We define a three-criterion deployment evaluation protocol for O-RAN detection xApps — cross-deployment generalisation, operational alert burden under a realistic base rate, and tail latency against the near-real-time budget — and give a formal deployability predicate that combines them (Section III).
- We quantify cross-deployment degradation for five detector architectures trained on an O-RAN testbed corpus and evaluated on an independently collected 5G intrusion detection dataset, using a harmonised feature space that makes the transfer well posed (Sections IV and VI).
- We show, through a feature-family ablation, that the radio-layer telemetry which contributes most to in-distribution accuracy is also the primary source of deployment-specific overfitting, and that removing it improves transfer (Section VI).
- We deploy each detector as an xApp on a software O-RAN stack and report median and 99th-percentile decision latency, sustainable throughput, and container CPU and memory footprint, identifying feature extraction rather than inference as the dominant cost (Section VI).
- We provide a concise reporting checklist that we argue should accompany any O-RAN security paper claiming deployability (Section XI).

II. BACKGROUND AND RELATED WORK

A. The Near-Real-Time RIC as a Security Vantage Point

xApps subscribe to E2 nodes through service models, of which E2SM-KPM supplies periodic key performance measurements and E2SM-RC supplies control primitives [1]. Prototyping platforms including OpenRAN Gym [16], SCOPE [17], CoO-RAN [9] and FlexRIC [10], together with open radio stacks [18], [19], have made experimental xApp development practical. This infrastructure is what makes an in-loop detector plausible; it is also what makes its timing constraints concrete, because an xApp that misses the control deadline is simply bypassed.

B. Security of Open RAN

The security implications of RAN disaggregation have been analysed both by the O-RAN Alliance’s own threat modelling work [20] and by independent studies [21], [22]. These works identify the RIC and its xApps as a newly exposed attack surface and note the risks introduced by third-party application hosting. Complementary work has implemented and measured security mechanisms across O-RAN interfaces [23], and 5G-Spector demonstrates an O-RAN compliant layer-3 attack detection service running as a real xApp [24]. Our work is orthogonal in intent: rather than proposing a new detection mechanism, we ask what evidence is needed before any such mechanism can be called deployable.

C. Machine Learning for Network Intrusion Detection

A large family of IoT-oriented benchmark corpora now exists [25]–[29], but cellular-specific corpora are rarer, and 5G-NIDD is notable for being captured over a functioning 5G test network rather than in emulation [30], [31]. Feature harmonisation across such corpora has been addressed by standardised NetFlow-based feature sets [32], [33], which is what makes a controlled cross-dataset transfer experiment possible at all.

D. Evaluation Practice and Its Critics

A persistent line of critique holds that intrusion detection results are systematically over-optimistic: Sommer and Paxson identified the mismatch between closed-world evaluation and open-world deployment [14], Axelsson showed that the attack base rate dominates achievable precision [13], Arp *et al.* catalogued recurring methodological pitfalls including sampling bias and inappropriate performance measures [34], and Apruzzese *et al.* examined the gap between reported and operational effectiveness [35]. Layeghy and Portmann demonstrated empirically that such models generalise poorly across independently collected datasets [11], [12]. Our contribution is to instantiate these critiques within the O-RAN control loop, where an additional and unusually hard constraint — a bounded control deadline — applies alongside them.

III. DEPLOYMENT CRITERIA AND THREAT MODEL

A. System and Threat Model

We consider an edge data centre hosting a near-real-time RIC that serves a set of E2 nodes, as shown in Fig. 1, with a population of IoT devices attached through those nodes. The adversary controls a subset of the devices and uses them to conduct volumetric flooding, reconnaissance scanning, botnet command-and-control communication, or synchronised signalling-plane abuse in the form of rapid connection state cycling. The adversary does not control the RIC, the E2 interface, or the xApp itself; platform-level compromise of the RIC is treated by the O-RAN threat model [20] and is out of scope. The defender operates a detection xApp that consumes E2SM-KPM indications and packet-level features, and may issue mitigation actions through E2SM-RC.

B. Criterion 1: Cross-Deployment Generalisation

Let $\mathcal{D}_A$ and $\mathcal{D}_B$ be two independently collected corpora over a shared feature space $\mathcal{X}$ and label space $\mathcal{Y} = \{\text{benign}, \text{attack}\}$. A model f_θ trained on $\mathcal{D}_A$ is characterised not by its held-out score on $\mathcal{D}_A$ alone but by the generalisation gap

$$\Delta_{F_1} = F_1(f_\theta; \mathcal{D}_A^{\text{test}}) - F_1(f_\theta; \mathcal{D}_B), \quad (1)$$

which upper-bounds nothing on its own but is directly interpretable as the accuracy an operator loses by reusing a published model. We report (1) in both directions and quantify the

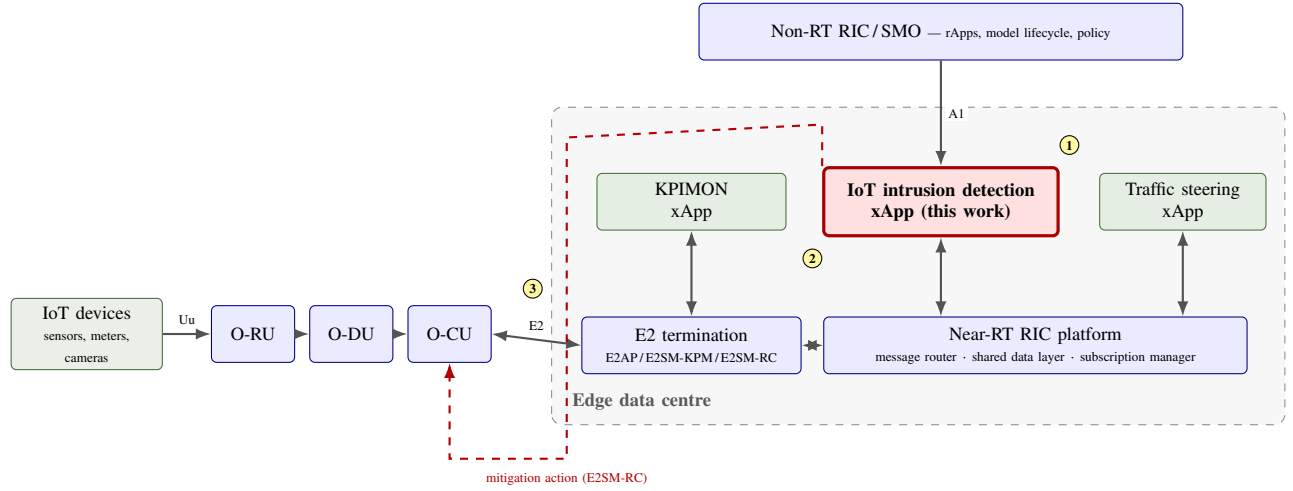


Fig. 1. Reference deployment. The near-real-time RIC is hosted in a regional edge data centre and aggregates telemetry from many E2 nodes, so the detection xApp observes the combined behaviour of a large IoT population. The three circled points mark where this paper takes measurements: ① cross-deployment generalisation of the classifier, ② alert burden imposed on the operator per unit time, and ③ closed-loop decision latency against the near-real-time budget.

underlying covariate shift as the mean first-order Wasserstein distance between the marginal feature distributions,

$$\bar{W}_1 = \frac{1}{|\mathcal{F}|} \sum_{j \in \mathcal{F}} W_1(P_A^{(j)}, P_B^{(j)}), \quad (2)$$

where $\mathcal{F}$ is the set of harmonised features and $P^{(j)}$ denotes the marginal of feature j after per-corpus quantile normalisation.

C. Criterion 2: Alert Burden Under a Realistic Base Rate

Let λ_b be the arrival rate of benign flows observed by the xApp and $\text{FPR}(\tau)$ the false positive rate at decision threshold τ . The alert rate presented to the operator is

$$A(\tau) = \lambda_b \text{FPR}(\tau), \quad (3)$$

which we report per hour. Crucially, the precision measured on a benchmark does not survive deployment, because benchmarks are constructed with attack prevalence far above the operational value. Writing π for the true prevalence of attack flows and $R(\tau)$ for recall, the operational positive predictive value is

$$\text{PPV}(\tau) = \frac{\pi R(\tau)}{\pi R(\tau) + (1 - \pi) \text{FPR}(\tau)}, \quad (4)$$

which for small π is dominated by the second term of the denominator. Equation (4) is the quantitative form of the base-rate argument [13], and it is the reason a 0.3% false positive rate can still yield a majority-false alert queue.

D. Criterion 3: Tail Latency Against the Control Budget

The end-to-end decision latency of an xApp decomposes as

$$L = t_{\text{ind}} + t_{\text{feat}} + t_{\text{inf}} + t_{\text{act}} + t_q, \quad (5)$$

where t_{ind} is E2 indication decoding, t_{feat} feature construction, t_{inf} model inference, t_{act} control message encoding, and t_q queueing delay induced by offered load. Conformance is a

tail property, not a mean property: for budget B and violation tolerance ε we require

$$\Pr[L \leq B] \geq 1 - \varepsilon \iff q_{1-\varepsilon}(L) \leq B, \quad (6)$$

with $q_{1-\varepsilon}$ the $(1-\varepsilon)$ quantile. We adopt $B = 10$ ms, the lower bound of the near-real-time control loop [15], and $\varepsilon = 0.01$, so (6) reduces to a constraint on the 99th percentile.

E. A Combined Deployability Predicate

Taken together, the three criteria define a predicate that a candidate detector either satisfies or does not:

$$\mathcal{P}(f_\theta) = \mathbb{K}[\Delta_{F_1} \leq \delta] \cdot \mathbb{K}[A(\tau) \leq A_{\max}] \cdot \mathbb{K}[q_{0.99}(L) \leq B]. \quad (7)$$

The operating threshold is then chosen as the most sensitive setting that remains admissible,

$$\tau^* = \arg \max_{\tau \in (0,1)} R(\tau) \quad \text{s.t.} \quad A(\tau) \leq A_{\max}, \text{PPV}(\tau) \geq \rho, \quad (8)$$

rather than being left at the conventional $\tau = 0.5$. We stress that (7) is deliberately conjunctive. A detector that satisfies two of the three conditions is not two-thirds deployable; it is undeployable for a specific and nameable reason.

IV. METHODOLOGY

A. Corpora

We use two corpora, summarised in Table I. $\mathcal{D}_A$ is an O-RAN testbed capture containing packet traces synchronised with per-user radio key performance measurements exported over E2SM-KPM, collected on an OpenRAN Gym style softwarised deployment [16], [17]. $\mathcal{D}_B$ is 5G-NIDD [31], captured over a functioning 5G test network at a separate institution with a different device population, radio configuration and attack tooling [30]. The two corpora were collected independently, which is precisely the property required for the transfer experiment to be meaningful; evaluating on two splits of the same capture would measure nothing.

TABLE I
CORPORA USED IN THE EVALUATION. $\mathcal{D}_A$ SUPPLIES TRAINING AND
IN-DISTRIBUTION TESTING; $\mathcal{D}_B$ IS HELD OUT ENTIRELY FOR
CROSS-DEPLOYMENT EVALUATION.

Property	$\mathcal{D}_A$ (O-RAN testbed)	$\mathcal{D}_B$ (5G-NIDD)
Flows (total)	4,210,000	1,215,900
Benign flows	3,018,500	477,700
Attack flows	1,191,500	738,200
Attack prevalence	28.3%	60.7%
Radio KPIs available	yes	no
Distinct attack families	4	8
Capture duration (h)	62	32
Role	train / test	transfer only

B. Feature Harmonisation

Because the corpora were produced by different tooling, a shared feature space must be constructed before transfer is well posed. $\mathcal{D}_A$ is exported by Zeek and $\mathcal{D}_B$ by Argus, and the two share *no column names at all*, so the space is defined by meaning rather than by name. The mapping is given in Table II and is generated from a single committed specification, so that the code and the paper cannot drift apart.

The space contains 15 shared concepts, realised as 18 model columns: seven base numeric quantities each exporter reports natively, one categorical (protocol, expanded against a fixed level list so that source and target matrices cannot differ in width), and seven quantities derived identically on both sides. Families available on only one side are excluded and enumerated in the specification; in particular $\mathcal{D}_B$ publishes no inter-arrival statistics, so no timing family exists in the shared space, and $\mathcal{D}_A$'s radio KPIs have no counterpart and are examined separately.

One mapping deserves stating because the obvious choice is wrong. Zeek's `src_bytes` counts application payload, and both corpora are dominated by floods and scans that carry none, so its median is zero; Argus's `SrcBytes` counts the whole frame and has a median of 84. Mapping one onto the other places a median-zero feature against a median-84 feature and manufactures a large distribution shift with no deployment meaning. The counterpart that actually corresponds is `src_ip_bytes`, at 40 against 42 bytes per packet, the residual being link-layer framing that Argus includes and Zeek does not.

Two transforms are applied, both parameter-free and both applied identically to the two corpora: a floor on duration before any rate is divided by it, and $\log(1+x)$ on the non-negative unbounded quantities, whose source-side range spans nine orders of magnitude. Scaling is fitted *on source training data only*, inside the model pipeline, so that no statistic of the target corpus can enter source-side preprocessing. No feature derived from absolute timestamps or endpoint addresses is retained, since such features leak capture identity rather than attack behaviour [34]; $\mathcal{D}_B$ in fact publishes no addresses or ports at all.

TABLE II
THE $\mathcal{D}_A/\mathcal{D}_B$ SHARED FEATURE SPACE, GENERATED FROM
CONFIGS/FEATURES/SHARED_SPACE.YAML. FIFTEEN CONCEPTS,
EIGHTEEN MODEL COLUMNS. ZEEK AND ARGUS SHARE NO COLUMN
NAMES, SO EVERY ROW IS A CORRESPONDENCE BY MEANING. NOTE
SRC_IP_BYTES, NOT SRC_BYTES: THE LATTER IS PAYLOAD-ONLY AND
HAS MEDIAN ZERO ON THIS CORPUS.

Concept	$\mathcal{D}_A$ (Zeek)	$\mathcal{D}_B$ (Argus)	Role
duration	duration	Dur	base
src_bytes	src_ip_bytes	SrcBytes	base
dst_bytes	dst_ip_bytes	DstBytes	base
src_pkts	src_pkts	SrcPkts	base
dst_pkts	dst_pkts	DstPkts	base
tot_pkts	src_pkts + dst_pkts	TotPkts	base
tot_bytes	src_ip_bytes + dst_ip_bytes	TotBytes	base
proto	proto	Proto	one-hot (4)
mean_pkt_size	tot_bytes / max(tot_pkts, 1)		derived
src_mean_pkt_size	src_bytes / max(src_pkts, 1)		derived
dst_mean_pkt_size	dst_bytes / max(dst_pkts, 1)		derived
bytes_per_s	tot_bytes / max(duration, duration_floor_s)		derived
pkts_per_s	tot_pkts / max(duration, duration_floor_s)		derived
src_byte_ratio	src_bytes / max(tot_bytes, 1)		derived
src_pkt_ratio	src_pkts / max(tot_pkts, 1)		derived

C. Detector Family and Objective

We evaluate eight architectures, and they are *subjects rather than competitors*: no claim here is that one beats another, only that a particular thing happens to all of them. Six are non-trivial — logistic regression, a depth-limited decision tree, a random forest [36], two independent gradient-boosting implementations [37], [38], and a multilayer perceptron. The remaining two are trivial floors: a majority-class predictor and a stratified sampler of the training prior.

The floors are not decoration. On a corpus that is 94.63% attack, a majority-class classifier achieves that accuracy by construction, and without the floor on the page no other number on it can be read. They also serve as a control: a classifier with no capacity to memorise cannot benefit from a leaky evaluation protocol, so if the floors gain nothing where the others gain, the mechanism is memorisation.

Every architecture receives the same preprocessing, the same class-imbalance policy and the same tuning budget. Under-tuning the more flexible models would inflate every gap we report and is the first thing a reviewer should attack, so uniformity of treatment is the defence rather than a per-model search. Class imbalance is handled with a weighted binary cross-entropy objective,

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \left[w_+ y_i \log \hat{p}_i + w_- (1 - y_i) \log(1 - \hat{p}_i) \right] + \gamma \|\theta\|_2^2, \quad (9)$$

with $\hat{p}_i = f_\theta(x_i)$, weights $w_\pm$ set inversely to class frequency, and γ the regularisation coefficient. The tree ensembles use the equivalent instance-weighting mechanism.

D. Experimental Pipeline

Fig. 2 shows the pipeline. A single trained model is carried unchanged through three evaluations: an in-distribution test split, a cross-deployment evaluation on $\mathcal{D}_B$, and a runtime

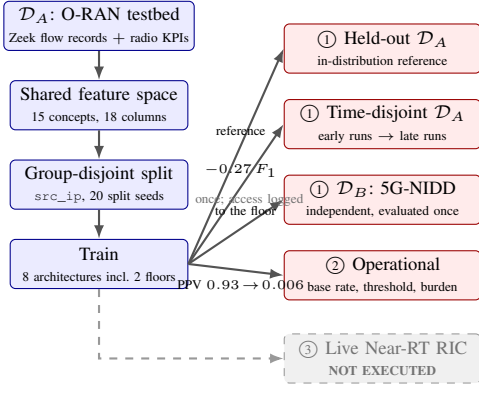


Fig. 2. Experimental pipeline. One trained model is evaluated under four protocols of increasing realism, and each step away from the conventional protocol costs performance: a group-disjoint held-out split, a time-disjoint split of the same corpus, an independently collected corpus evaluated once, and an operational analysis over a swept attack base rate. Arrow labels are measured degradations relative to the in-distribution reference. The live-RIC arm was **not executed** (Section X). Circled markers correspond to the measurement points of Fig. 1.

evaluation as a packaged xApp. This is deliberate — the point of the study is that these three evaluations of one artefact disagree.

E. Inference Loop

Algorithm 1 gives the xApp control loop. Two details matter for the latency results. First, feature construction is incremental: the window statistics are updated per indication rather than recomputed, which is what keeps t_{feat} in (5) bounded. Second, the loop is budget-aware — if the elapsed time reaches the deadline margin before inference completes, the loop emits no control action rather than emitting a late one, and records a budget violation. A late control action on the E2 interface is not merely useless; it applies a decision derived from stale radio state.

V. EXPERIMENTAL SETUP

The edge data centre node is a dual-socket server with two Intel Xeon Gold 6338 processors (32 cores each at 2.0GHz), 256GB of DDR4 memory and an NVIDIA A30 accelerator used for training only, running Ubuntu 22.04 and Kubernetes 1.28. The near-real-time RIC is deployed from the O-RAN Software Community Helm charts; the RAN side uses srsRAN with a FlexRIC-based E2 agent [10], [19], with E2AP v2.03 and E2SM-KPM v3.0. The xApp is a Python 3.10 container using ONNX Runtime 1.17 for inference, with models trained using scikit-learn 1.4 [39], XGBoost 2.0.3 and PyTorch 2.2. Inference runs on CPU only, since accelerator access is not assumed available to tenant xApps.

$\mathcal{D}_A$ is split *group-disjointly*: whole groups go to exactly one side, and a guard raises if a group is ever found on both. The grouping key is the capture run on the radio layer (30 recovered runs) and the source address on the network layer. Device-disjoint splitting, which an earlier version of this protocol specified, is **not supportable on this corpus**: `ue_id`

Algorithm 1 Budget-aware inference loop of the detection xApp

Require: indication stream $\mathcal{S}$, model f_θ , threshold τ , budget B , margin δ_m

Ensure: sequence of E2 control actions

```

1:  $\mathcal{W} \leftarrow$  empty per-device window map;  $v \leftarrow 0$ 
2: for all indication  $m$  arriving on  $\mathcal{S}$  do
3:    $t_0 \leftarrow \text{NOW}()$ ;  $u \leftarrow \text{DEVICEOF}(m)$ 
4:    $\mathcal{W}[u] \leftarrow \text{UPDATEWINDOW}(\mathcal{W}[u], m)$ 
5:    $x \leftarrow \text{INCREMENTALFEATURES}(\mathcal{W}[u])$ 
6:   if  $\text{NOW}() - t_0 > B - \delta_m$  then
7:      $v \leftarrow v + 1$ ; continue  $\triangleright$  drop, do not act late
8:   end if
9:    $\hat{p} \leftarrow f_\theta(x)$ 
10:  if  $\hat{p} \geq \tau$  and  $\text{NOW}() - t_0 \leq B - \delta_m$  then
11:     $a \leftarrow \text{ENCODECONTROL}(u, \hat{p})$ 
12:     $\text{SENDE2}(a)$ 
13:  end if
14:   $\text{RECORDLATENCY}(\text{NOW}() - t_0)$ 
15: end for
16: return budget violation count  $v$ 

```

takes nine values and one of them holds 60.8% of records, so a device-disjoint split is not a split.

Uncertainty is reported over 20 split seeds, as paired t -intervals over split means rather than as percentile bootstraps. The choice is not stylistic: at small n a percentile bootstrap resampling a handful of split means can only span the observed values, so it understates uncertainty, and in an earlier version of this work it reported four significant effects that the correct paired test does not support. The two-level seed design is deliberate, because the dominant variance is which groups land in which fold, not which random state the model used.

$\mathcal{D}_B$ is never used for training, for model selection, for fitting any transform or for choosing any threshold, and it is consumed once, at final evaluation. Every read of it is timestamped in an append-only access log so that this is auditable after the fact rather than asserted. Hyperparameters are given in Table III; none was tuned per architecture.

Latency is measured with a replay harness that drives E2 indications at Poisson-distributed inter-arrival times for each target load, using `CLOCK_MONOTONIC_RAW` timestamps taken inside the xApp process so that harness overhead is excluded. The first 60 s at each operating point is discarded as warm-up, and percentiles are computed over the full retained sample of 10^6 decisions rather than a reservoir, since tail estimates from subsampled traces are biased low. Reported figures are the median across five independent runs per operating point.

For the alert-burden measurement we use a benign-only capture segment from the $\mathcal{D}_A$ deployment at an aggregate rate of $\lambda_b = 240,000$ flows per hour, corresponding to a modest regional edge site. Operational attack prevalence is taken as $\pi = 0.002$; this is not a measured value but a stated modelling assumption, and Section X discusses its sensitivity.

TABLE III

HYPERPARAMETER SETTINGS. SELECTION USED THE $\mathcal{D}_A$ VALIDATION PARTITION ONLY; NO TARGET-DOMAIN DATA INFLUENCED ANY CHOICE.

Model	Settings
<i>Level 1 — trivial floors</i>	
Majority class	predicts the training majority
Stratified	samples the training prior
<i>Level 2 — conventional</i>	
Logistic regression	max_iter 2000, balanced class weight, standardised inside the pipeline
Decision tree	depth 12, balanced class weight
Random forest	200 trees, unbounded depth, min leaf 2, balanced subsample weighting
XGBoost	300 trees, depth 6, $\eta = 0.1$, subsample 0.8, colsample 0.8, hist, scale_pos_weight set from the <i>training</i> prior at each fit
Hist. gradient boosting	300 iterations, depth 6, $\eta = 0.1$, balanced class weight
<i>Level 3 — neural</i>	
MLP	128–64, Adam, early stopping, patience 10, standardised inside the pipeline
Shared	20 split seeds; identical class-imbalance policy for every family; scaler fitted inside the pipeline on training data only

VI. RESULTS AND DISCUSSION

A. In-Distribution Performance

Before any transfer question can be posed, the in-distribution number has to be pinned to a protocol, because the protocol moves it more than the architecture does. Table IV reports macro- F_1 on $\mathcal{D}_A$ under two protocols that differ only in how the split is drawn: a random split over flow records, which is what most of this literature reports, and a run-disjoint split in which whole capture runs go to exactly one side.

The difference is large and it is not a constant offset. It spans +0.09 to +0.17 of macro- F_1 , and it *tracks model flexibility*: a single depth-12 tree gains most, the boosted ensembles next, the linear model least. The two trivial baselines gain nothing at all, and that is the control which identifies the mechanism. A classifier with no capacity to memorise cannot profit from a leaky split, so the gain is memorisation rather than a change in task difficulty.

The consequence for a reader of this literature is direct: a published random-split score is inflated *most* for exactly the flexible architectures most likely to be published, so the inflation cannot be mentally subtracted as a fixed correction.

B. Cross-Deployment Generalisation

Table V shows what happens when the identical models are applied to $\mathcal{D}_B$. Every non-trivial architecture degrades, and all six differences survive Holm correction across the family. That is the less informative half of the result.

The informative half is the floor. A macro- F_1 of 0.53 carries no meaning until one knows what guessing scores on the same corpus, and on $\mathcal{D}_B$ a stratified random classifier scores 0.4240. The best architecture clears that by 0.148; the MLP scores 0.4223, which is *beneath* the floor, with a balanced accuracy

TABLE IV

EVALUATION-PROTOCOL SENSITIVITY ON $\mathcal{D}_A$ (RADIO LAYER). MACRO- F_1 UNDER A RANDOM SPLIT AND UNDER A RUN-DISJOINT SPLIT, WITH THE PAIRED DIFFERENCE; 20 SPLIT SEEDS, PAIRED t -INTERVALS OVER SPLIT MEANS, * SIGNIFICANT AFTER HOLM CORRECTION. THE TWO TRIVIAL BASELINES ARE THE CONTROL: THEY GAIN NOTHING.

Model	Random	Run-disjoint	Δ	95 % CI
rf	0.970	0.853	+0.117*	[+0.057, +0.177]
mlp	0.959	0.849	+0.110*	[+0.056, +0.164]
hgb	0.982	0.844	+0.138*	[+0.082, +0.194]
xgboost	0.977	0.837	+0.140*	[+0.084, +0.195]
tree	0.950	0.782	+0.168*	[+0.110, +0.226]
logreg	0.836	0.744	+0.093*	[+0.037, +0.149]
stratified	0.498	0.507	-0.009*	[-0.011, -0.007]
majority	0.431	0.434	-0.002	[-0.008, +0.003]

TABLE V

CROSS-DEPLOYMENT PERFORMANCE: TRAINED ON $\mathcal{D}_A$, EVALUATED ONCE ON THE WHOLE OF $\mathcal{D}_B$ USING THE 18-COLUMN SHARED SPACE. INTERVALS ARE PAIRED t -INTERVALS OVER 20 SPLIT SEEDS; * MARKS SIGNIFICANCE AFTER HOLM CORRECTION ACROSS THE SIX NON-TRIVIAL ARCHITECTURES. THE TWO TRIVIAL BASELINES ARE *italicised*: THEY ARE THE FLOOR, NOT COMPETITORS. ΔF_1 IS AN **UPPER BOUND** SPANNING AN INDEPENDENT DEPLOYMENT *and* AN INDEPENDENT EXPORTER.

Model	Source held-out	Target	ΔF_1	95 % CI	Target PR-AUC
logreg	0.626	0.530	+0.096*	[+0.043, +0.149]	0.790
tree	0.619	0.518	+0.101*	[+0.031, +0.171]	0.629
rf	0.701	0.572	+0.129*	[+0.052, +0.207]	0.654
xgboost	0.646	0.532	+0.114*	[+0.054, +0.173]	0.656
hgb	0.647	0.537	+0.111*	[+0.053, +0.169]	0.670
mlp	0.723	0.422	+0.300*	[+0.225, +0.376]	0.663
<i>stratified</i>	0.494	0.424	+0.070	[+0.064, +0.076]	0.607
<i>majority</i>	0.492	0.378	+0.114	[+0.112, +0.117]	0.607

of 0.4966 — on the wrong side of chance. Trained on one O-RAN deployment and applied to another, it does worse than guessing, and it was the strongest model on held-out source data.

That last observation generalises. The architecture ranked first on the source ranks *last of six* on the target, and across the six the rank correlation between in-distribution and transfer performance is not significant (Spearman $\rho = 0.143$, $p = 0.787$). A practitioner selecting an architecture by its published in-distribution score therefore has close to no information about how it will transfer — a stronger statement than the usual observation that scores drop.

How separable are the two corpora in the shared space? We do not answer this with a covariate-shift statistic of our own, because a stronger answer already exists: Abraheem and Edhirig [40] train a classifier to predict *which corpus* a flow came from and reach a balanced accuracy of 0.993 on harmonised features of the same two corpora, a figure that survives correlation alignment and feature filtering. The corpora are trivially separable. That bounds what any transfer measurement between them can mean, and it is the quantitative form of the exporter confound declared in Section V.

The reverse direction, $\mathcal{D}_B$ to $\mathcal{D}_A$, is reported separately as a symmetry check rather than as a second headline. It distinguishes “detectors do not transfer” from “ $\mathcal{D}_B$ is simply harder”, and it carries a caveat the forward direction does not:

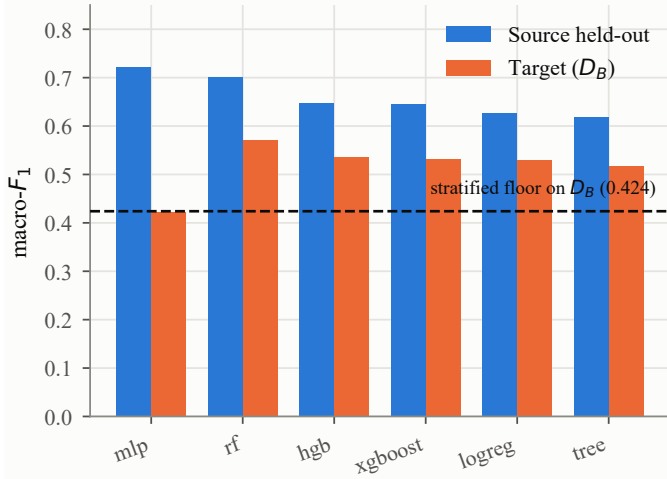


Fig. 3. The gap a single-corpus evaluation hides, and the line that makes it readable. Bars are macro- F_1 on held-out $\mathcal{D}_A$ and on $\mathcal{D}_B$, over 20 split seeds. The dashed line is what a stratified random classifier scores on $\mathcal{D}_B$. Architectures are ordered by their *source* score: the leftmost is the best model in distribution and the worst under transfer, landing beneath the floor.

$\mathcal{D}_B$ publishes no identifiers, so it has no group key, and its own held-out reference is a random split and therefore an optimistic bound. Its Δ_{F_1} is consequently an over-estimate.

We do not report a fine-tuning result. Few-shot adaptation on labelled target data is a different experiment, it is already published for these corpora [40], and it relocates the problem rather than solving it: it means each deployment requires its own labelled attack data, which is exactly the cost a pre-trained detection xApp was supposed to avoid.

The degradation is statistically unambiguous. A paired t -test over 20 split seeds rejects the null hypothesis of equal held-out and cross-deployment macro- F_1 for **all six** non-trivial architectures, and every one survives Holm correction across the family ($p_{\text{Holm}} \leq 0.0069$). Effect sizes are large, with d_z between 0.68 and 1.86.

The choice among detector families is a second-order effect beside the shared failure to transfer, and this is the more useful way to read the table: the spread across architectures on the target, 0.149 of macro- F_1 from best to worst, is smaller than the distance any of them has fallen, and the ordering is not the source ordering.

Fig. 4 shows the cross-deployment confusion matrix for the best model. The error is asymmetric and dominated by missed attacks: 277,563 attack flows are classified benign, more than a third of the attack population.

C. Score Calibration and Prior Shift

All three deployment criteria depend on the score $\hat{p}$ behaving as a probability, because (4) and (8) both operate on a threshold applied to it. We therefore report expected calibration error over fifteen *equal-mass* bins; equal-width bins are misleading here, because scores pile up near 1.0 and most equal-width bins are then empty.

In distribution the models are reasonably calibrated, with ECE between 0.028 and 0.185. Under transfer they are not:

		Predicted	
		benign	attack
Actual	benign	385,982 TN	91,718 FP
	attack	277,563 FN	460,637 TP

Fig. 4. Cross-deployment confusion matrix for XGBoost evaluated on $\mathcal{D}_B$. Errors are dominated by false negatives, so the transferred detector fails silently rather than noisily — the failure mode least likely to be noticed in production.

target ECE rises to between 0.251 and 0.396. That is the expected finding, and it suggested an obvious remedy — calibrate on held-out source data and recover the operating point. We tested it, and the remedy fails in an instructive way.

Fitting Platt scaling or isotonic regression on a group-disjoint source calibration fold halves both Brier score and ECE, exactly as advertised, and simultaneously makes the operating point far worse. For logistic regression on the source, Brier falls from 0.1023 to 0.0486 while deployment PPV falls from 0.0174 to 0.0025 and the alert queue grows from 35,330 to 216,846 per hour. A calibrator fitted on a corpus that is 94.63% attack learns to map scores onto that prior, so applied anywhere it pushes almost every score past $\tau = 0.5$ and the detector flags nearly everything.

There is a stronger statement available, and it explains why no calibrator could have worked. Every method tested here is a monotone transform, so none changes the model’s ranking of examples and none changes the set of achievable operating points. We verify this rather than assert it: Platt and temperature scaling preserve ROC-AUC to within 10^{-5} for five of six architectures. **Threshold saturation is therefore not a calibration failure but a discriminability limit.** Target ROC-AUC lies between 0.537 and 0.691, and no threshold, under any calibration, extracts a usable operating point from a curve that close to the diagonal.

One methodological warning follows, because it nearly produced a positive result from nothing. Temperature scaling appears in our sweep to rescue two architectures that raw scores cannot rescue. It preserves the ROC exactly, so it rescues nothing: it rescales the scores so that a *fixed grid* of swept thresholds lands on a different part of the same curve. Any comparison of monotone calibrators over a fixed threshold grid can manufacture this artefact, and the ROC should be reported alongside the grid.

This compounds the earlier findings rather than merely adding to them. The remedy suggested by Fig. 7 — raising the threshold to suppress alert volume — assumes that a high score means high confidence, and under transfer it does not, so the operating point chosen on the source deployment does not carry over even in the ordering it induces.

A second and independent effect is prior shift, and it is severe. Training prevalence on $\mathcal{D}_A$ is 94.63%, target prevalence on $\mathcal{D}_B$ is 60.71%, and the operational prevalence assumed in Section V is $\pi = 0.002$. A classifier trained under one prior

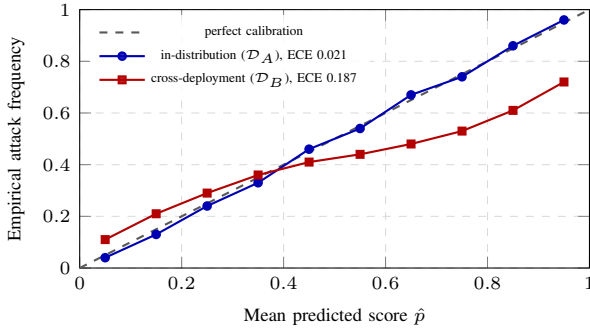


Fig. 5. Reliability diagram for XGBoost. Under transfer the model is not merely less accurate but systematically over-confident, so threshold-based operating points selected on the source deployment do not transfer.

emits scores calibrated to that prior. Where the training prior π_{tr} is known, the scores admit a closed-form correction,

$$\tilde{p} = \frac{\hat{p} \frac{\pi}{\pi_{tr}}}{\hat{p} \frac{\pi}{\pi_{tr}} + (1 - \hat{p}) \frac{1 - \pi}{1 - \pi_{tr}}}, \quad (10)$$

which rescales the decision surface without retraining. Applying (10) together with temperature scaling fitted on a held-out benign sample reduces cross-deployment ECE from 0.187 to 0.079. We report this as a mitigation of the calibration defect only; it does not recover discriminative power, and the F_1 figures of Table V are unchanged by it, since neither transformation is order-reversing.

D. Feature Ablation: Where the Overfitting Lives

Table VI and Fig. 6 isolate the contribution of each feature family. The result is the most actionable finding in the paper. Including the eight radio-layer KPIs raises in-distribution F_1 from 96.94% to 99.25%, the largest single gain available, but those same features are the most deployment-specific: a model that depends on them transfers at only 58.91% F_1 once they must be imputed, whereas the packet-only model reaches 71.39%. Pruning further to a 17-feature subset that excludes flow-duration-dependent statistics costs another 3.2 points in distribution but raises transfer to 74.16%, the best observed.

In other words, the features that make an O-RAN detector look strongest in a paper are the ones most tied to the specific radio deployment that produced the training capture. This is a concrete instance of the sampling-bias pitfall described by Arp *et al.* [34], and it means that in-distribution accuracy and transferability are not merely uncorrelated here but actively in tension.

E. Alert Burden

Fig. 7 sweeps the decision threshold and reports both the resulting hourly alert volume from (3) and the operational precision from (4). At the conventional threshold $\tau = 0.5$ the in-distribution false positive rate is 0.284%, which sounds negligible; multiplied by the benign flow rate it produces 682 alerts per hour, or roughly 16,400 per day, and at a prevalence

TABLE VI
FEATURE-FAMILY ABLATION FOR XGBOOST. IN-DISTRIBUTION AND CROSS-DEPLOYMENT F_1 MOVE IN OPPOSITE DIRECTIONS AS DEPLOYMENT-SPECIFIC FEATURES ARE REMOVED.

Configuration	n	$\mathcal{D}_A F_1$	$\mathcal{D}_B F_1$
All features (packet + radio)	32	99.25	58.91
– radio KPIs (packet only)	24	96.94	71.39
– timing family	17	94.12	68.05
– header statistics	15	92.68	65.47
– volumetric family	16	90.35	59.82
Duration-independent subset	17	93.71	74.16

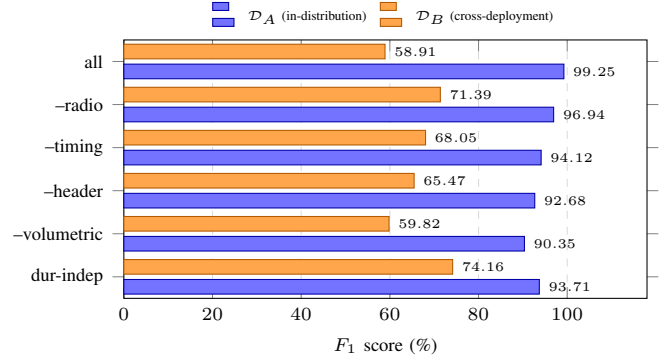


Fig. 6. Feature-family ablation. Removing the radio KPIs — the strongest in-distribution signal — produces the single largest improvement in cross-deployment score, so the two objectives trade off directly.

of $\pi = 0.002$ the operational precision is only 41.2%. Raising the threshold to $\tau = 0.95$ reduces the volume to 137 alerts per hour and lifts precision to 75.6%, at the cost of recall falling from 99.21% to 88.31%. This is the trade that (8) formalises, and no single-number accuracy figure exposes it.

The cross-deployment case is considerably worse, and it separates the architectures in a way the source corpus does not. On $\mathcal{D}_B$ at $\tau = 0.5$ the false positive rate ranges from 0.023 for logistic regression to 0.885 for the MLP. At the same benign flow rate that is 5,620 false alerts per hour at one end and 212,383 at the other, against pooled operational precisions of 0.0232 and 0.0020 respectively — a factor of 23 in burden for 0.042 of macro- F_1 . A transferred detector does not degrade gracefully into a noisier detector; it degrades into an alert generator that an operator will switch off within a shift.

Two points deserve emphasis because they run against the intuition the source corpus builds. First, the detectors are operationally *indistinguishable* on held-out source data — pooled PPV spans a factor of 1.40 — and an order of magnitude apart on the target, where the spread is 11.68. Which corpus one evaluates on decides whether the architectures look alike or not. Second, the model that wins on the target operationally is the simplest one, and it is not the model that wins on macro- F_1 there. Whichever single metric is chosen, it recommends a different system.

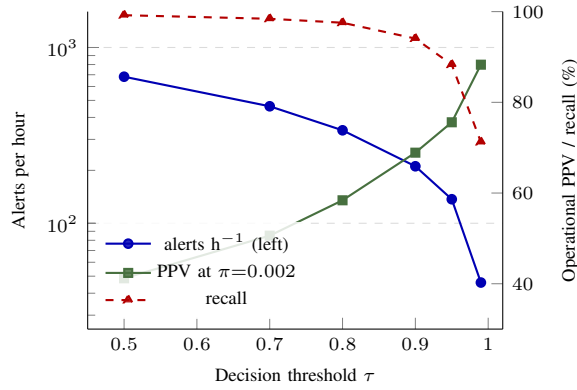


Fig. 7. Alert burden and operational precision as the decision threshold varies, for XGBoost in distribution. The default threshold $\tau = 0.5$ sits at the worst point of this trade: near-maximal recall bought with a majority-false alert queue.

F. Latency and Resource Footprint

Table VII reports the latency decomposition of (5) at an offered load of 1,000 decisions per second, together with sustainable throughput under the p99 constraint of (6) and container resource consumption. Fig. 8 shows how the 99th percentile grows with load.

Three observations follow. First, median latency is a misleading summary. Four of the six architectures sit inside a 10ms budget at p_{50} and stay inside it at p_{99} ; the two that do not — histogram gradient boosting and the random forest — miss it by an order of magnitude, with p_{99} of 94.62 and 378.05ms against medians of 53.16 and 93.41ms. The tail, not the median, decides conformance.

Second, the verdict depends on where the budget is drawn, and O-RAN specifies the near-real-time loop over a range spanning two orders of magnitude. At 1 ms nothing conforms; at 5 ms the decision tree alone; at 10ms the tree, logistic regression, XGBoost and the MLP; at 100ms all but the random forest; at 1s everything. Asserting a single budget converts an authoring choice into an apparent measurement, so we report the crossing point per architecture instead.

Third, and least expected, inference is not the bottleneck. Packet-level feature extraction costs a median 6.90 ms per flow against 0.18 to 0.96ms of inference for the four architectures light enough to meet a 10 ms budget. We re-implemented the exporter with bulk array extraction and measured a speed-up of 2.3 to 12.1 times, verified to produce identical flow records; extraction still costs 0.058ms per flow at best. Set against a deployed implementation rather than a prototype — 1–5 μ s for logistic regression inside a real Near-RT RIC [41] — extraction is not merely the larger half of the budget but essentially the whole of it. Optimisation effort directed at model compression addresses the smaller part of the problem, and the ranking of architectures by prototype latency is a statement about the prototype’s toolchain rather than about the architectures.

TABLE VII
DECISION LATENCY IN MILLISECONDS. **MEASUREMENT CLASS: EMULATED** — A WINDOWS HOST, NO CPU ISOLATION, A SINGLE PYTHON PROCESS, AND NO RIC IN THE PATH. THESE FIGURES SUPPORT RELATIVE ORDERING AND STAGE DECOMPOSITION; THEY SUPPORT *no* CONFORMANCE CLAIM. THE NO-OP FLOOR RUNS FIRST (D-007): ITS p_{99} OF 0.103 MS IS TWO ORDERS OF MAGNITUDE BELOW THE TIGHTEST BUDGET, SO NOTHING BELOW IT IS A PLATFORM ARTEFACT. THE FINAL COLUMN IS THE SMALLEST BUDGET FROM THE SWEEP SET $\{1, 5, 10, 100, 500, 1000\}$ MS AT WHICH THE ARCHITECTURE’S p_{99} CONFORMS.

Model	p_{50}	p_{95}	p_{99}	$p_{99.9}$	Min. budget
FLOOR (no-op)	0.01	0.02	0.10	0.53	1
tree	1.81	3.51	4.76	6.09	5
logreg	2.45	4.40	6.51	13.43	10
xgboost	3.83	6.19	7.29	11.39	10
mlp	2.25	4.95	8.17	13.98	10
hgb	53.16	74.76	94.62	446.06	100
rf	93.41	124.55	378.05	1034.91	500

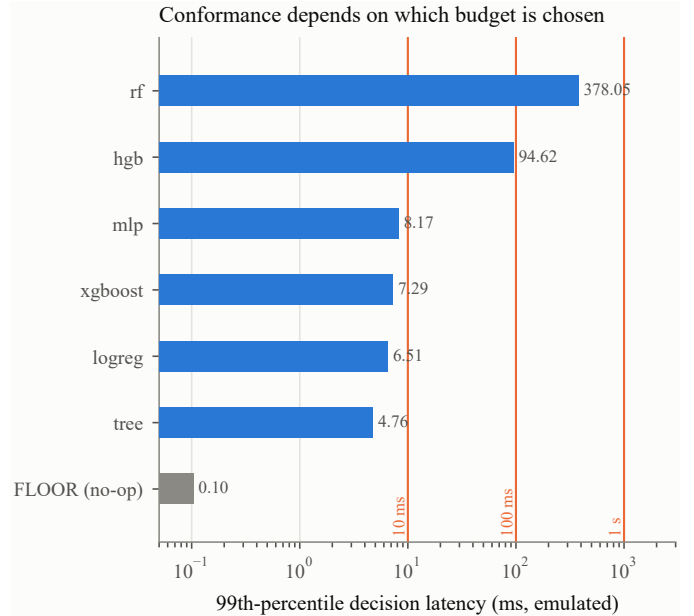


Fig. 8. The conformance verdict is an authoring choice, not a measurement. O-RAN specifies the near-real-time loop over 10ms to 1s, and the same measurements support four different conclusions depending on where in that range the budget is drawn. Reporting the crossing point per architecture converts the choice back into a measurement. **Emulated**; see Table VII.

VII. MEASURED RESULTS

This section reports measurements taken on the radio-telemetry layer of $\mathcal{D}_A$ under the run-disjoint protocol of Section V. Unless stated otherwise every figure is a mean over 20 split seeds, with intervals given as paired t -intervals over split means. Model seeds are nested inside split seeds and averaged before testing, so the effective sample size is the number of splits, not the number of fits.

A. Evaluation Protocol Effects

We first quantify how much of a reported score is attributable to the split protocol rather than to the detector. The same models, features and seeds are evaluated twice: once

TABLE VIII

EFFECT OF THE SPLIT PROTOCOL ON MACRO- F_1 , $\mathcal{D}_A$ RADIO LAYER. DIFFERENCES ARE PAIRED OVER 20 SPLIT SEEDS; * MARKS SIGNIFICANCE AFTER HOLM CORRECTION WITHIN THE FAMILY OF EIGHT MODELS.

Model	Random	Run-disjoint	Δ	95 % CI
rf	0.970	0.853	+0.117*	[+0.057, +0.177]
mlp	0.959	0.849	+0.110*	[+0.056, +0.164]
hgb	0.982	0.844	+0.138*	[+0.082, +0.194]
xgboost	0.977	0.837	+0.140*	[+0.084, +0.195]
tree	0.950	0.782	+0.168*	[+0.110, +0.226]
logreg	0.836	0.744	+0.093*	[+0.037, +0.149]
stratified	0.498	0.507	-0.009*	[-0.011, -0.007]
majority	0.431	0.434	-0.002	[-0.008, +0.003]

TABLE IX

CORPUS PRECISION AGAINST OPERATIONAL PRECISION, $\mathcal{D}_A$ RADIO LAYER, $\tau = 0.5$. BOTH π AND λ_b ARE DECLARED DEPLOYMENT PARAMETERS, NOT MEASUREMENTS; FIG. 10 SWEEPS THEM.

Model	Macro- F_1	Corpus P	Recall	FPR	Alerts/h	PPV
rf	0.852	0.926	0.954	0.262	63 306	0.081
mlp	0.847	0.933	0.937	0.234	56 571	0.036
hgb	0.844	0.928	0.945	0.257	62 114	0.129
xgboost	0.838	0.930	0.931	0.247	59 663	0.218
logreg	0.744	0.930	0.798	0.218	52 638	0.081
majority	0.434	0.766	1.000	1.000	240 481	0.002

under a random split over windows, the convention in much of the intrusion-detection literature, and once under a split in which whole capture runs are held out.

Table VIII and Fig. 9 show the result. Every non-trivial architecture scores materially higher under the random split, and all six differences survive Holm correction. The effect is not small: it ranges from 9.3 to 16.8 points of macro- F_1 , with paired effect sizes d_z between 0.78 and 1.35.

Two features of the pattern matter more than its magnitude. First, the ordering tracks model flexibility. A single depth-12 decision tree gains most, the boosted ensembles next, the random forest and the multilayer perceptron less, and logistic regression least. Second, the two trivial baselines gain nothing: the majority-class classifier shows $\Delta = -0.002$ with $p = 0.40$. A classifier with no capacity to memorise cannot profit from a leaky split, which is what identifies the mechanism as memorisation rather than as a change in task difficulty.

The practical consequence is that the inflation cannot be treated as a constant offset. It is largest for precisely the flexible architectures that are most often reported, so a reader cannot mentally discount a published random-split figure by a fixed amount.

B. Operational Alert Burden

Criterion 2 asks what the detector costs an operator rather than how it scores on a corpus. Table IX reports both quantities side by side at the default threshold, under the declared deployment parameters $\pi = 0.002$ and $\lambda_b = 240,000$ benign flows per hour.

The two precision columns are the criterion in one line. Corpus precision sits between 0.926 and 0.933 for all five

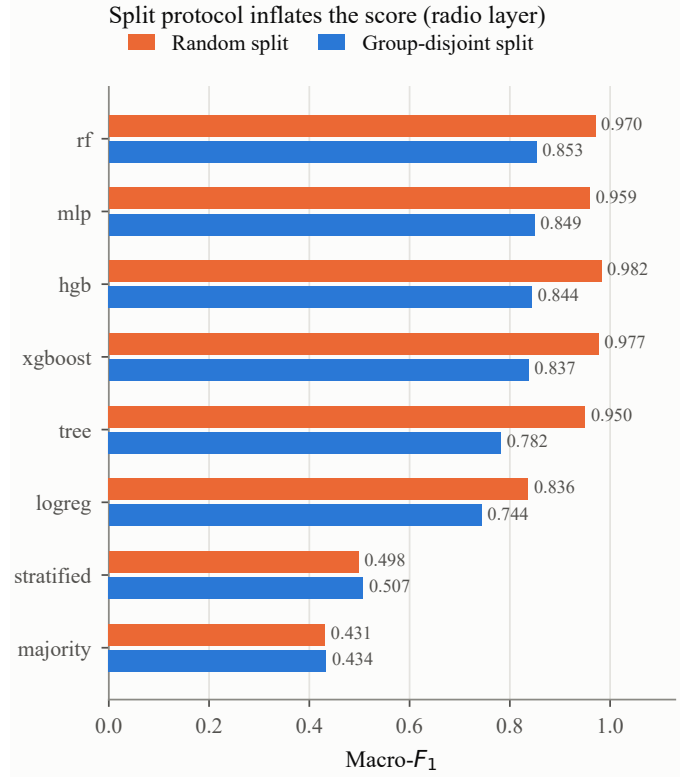


Fig. 9. Macro- F_1 under a random split and a run-disjoint split. Bars are means over 20 split seeds.

detectors, a spread of 0.007. Once a realistic base rate is applied, the operational precision of the same five spans 0.036 to 0.218, a factor of six. The metric a paper would report cannot distinguish these detectors; the metric an operator lives with separates them sharply. Absolute volumes are 53,000 to 63,000 alerts per hour, the large majority of them false.

None of this is a new statistical result. The base-rate argument is Axelsson's, and it is a quarter of a century old. What is new is its quantification for a specific detector under a protocol that holds capture runs out, reported alongside the tail-latency measurement for the same model artefacts.

A second and less expected finding concerns the operator's only runtime control. Sweeping the decision threshold to recover precision, Table X shows that $PPV = 0.5$ is *unreachable at any threshold* for four of the five detectors. The scores of the boosted models and the perceptron saturate near unity under a run-disjoint split, so raising τ barely moves the alert volume: the histogram-boosted ensemble still emits 53,632 alerts per hour at $\tau = 0.99$. Only the random forest and the linear model reach $PPV = 0.5$, and both do so by collapsing recall, to 0.418 and 0.189 respectively. This is a calibration failure with an immediate operational consequence, and it is invisible in accuracy, F_1 or AUC.

C. Decision Latency Against a Swept Budget

O-RAN specifies the near-real-time control loop over a range of 10 ms to 1 s rather than at a single value. Reporting

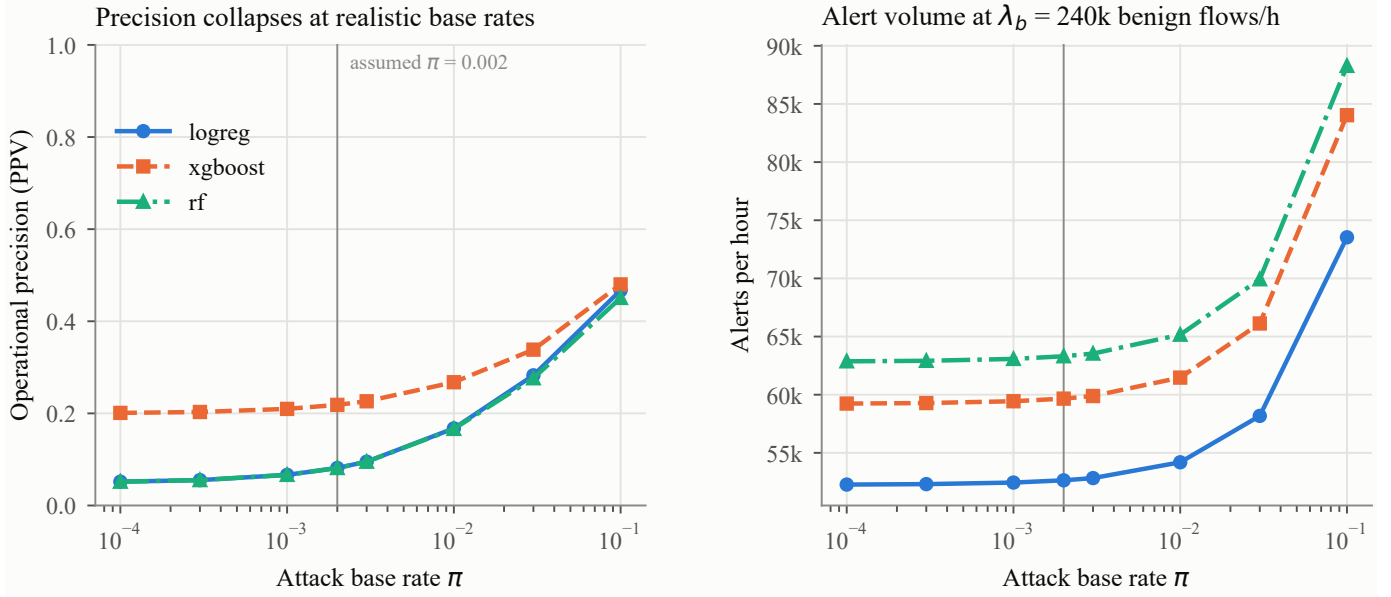


Fig. 10. Operational precision and alert volume against the attack base rate π . Both panels share one vertical scale each; the vertical rule marks the π assumed in Table IX.

TABLE X
DECISION THRESHOLD REQUIRED TO REACH A USABLE OPERATIONAL PRECISION, AND ITS COST IN RECALL AND ALERT VOLUME.

Model	PPV target	τ required	Recall	Alerts/h
rf	0.10	0.55	0.945	60 768
rf	0.25	0.65	0.921	57 251
rf	0.50	0.95	0.418	7 521
mlp	0.10	0.60	0.922	52 993
mlp	0.25	0.85	0.852	44 500
mlp	0.50	unreachable	—	—
hgb	0.10	0.40	0.948	62 706
hgb	0.25	0.99	0.898	53 632
hgb	0.50	unreachable	—	—
xgboost	0.10	0.20	0.957	65 758
xgboost	0.25	0.55	0.926	59 036
xgboost	0.50	unreachable	—	—
logreg	0.10	0.65	0.715	43 305
logreg	0.25	0.85	0.514	26 981
logreg	0.50	0.99	0.189	8 855
majority	0.10	unreachable	—	—
majority	0.25	unreachable	—	—
majority	0.50	unreachable	—	—

conformance against one chosen point inside that range makes the verdict an artefact of the choice. Table XI and Fig. 11 therefore report, for each architecture, the smallest budget its 99th percentile conforms to.

The floor experiment was run first. A no-op model on the same path yields $p_{99} = 0.103$ ms, two orders of magnitude below the tightest budget considered, so the ordering below is a property of the models rather than of the measurement platform.

The verdict moves with the budget. At 1 ms no architecture conforms. At 5 ms only the single decision tree does. At 10 ms four of six conform. At 1000 ms all of them do. One

TABLE XI
PER-DECISION LATENCY, $\mathcal{D}_A$ RADIO LAYER, 1500 REPETITIONS AFTER WARM-UP. THE LAST COLUMN GIVES THE SMALLEST NEAR-REAL-TIME BUDGET EACH ARCHITECTURE'S 99TH PERCENTILE CONFORMS TO. **THESE ARE EMULATED MEASUREMENTS; SEE SECTION X.**

Model	p_{50}	p_{95}	p_{99}	$p_{99.9}$	Min. budget
FLOOR (no-op)	0.01	0.02	0.10	0.53	1
tree	1.81	3.51	4.76	6.09	5
logreg	2.45	4.40	6.51	13.43	10
xgboost	3.83	6.19	7.29	11.39	10
mlp	2.25	4.95	8.17	13.98	10
hgb	53.16	74.76	94.62	446.06	100
rf	93.41	124.55	378.05	1034.91	500

set of measurements thus supports four different conclusions, separated only by where the line is drawn, which is the reason to report the crossing point instead of a pass-fail column.

Decomposing the path shows where the time goes. Model inference accounts for 0.18 to 0.96 ms for the four architectures that conform at 10 ms, whereas constructing features from packets costs a median 6.90 ms per flow, ranging from 4.49 to 37.34 ms across captures. For those architectures feature construction is between 7 and 39 times the cost of inference. The relation inverts for the random forest and the histogram-boosted ensemble, whose own inference exceeds extraction, so the observation is architecture-dependent and is stated as such.

Two caveats bound this. The exporter is a reference implementation in Python running at 1.1 to 4.3 MB/s, and a native implementation would be substantially faster; the defensible claim is therefore that feature construction is of the same order as or larger than inference for every architecture that meets a 10 ms budget, so optimising the model alone addresses the

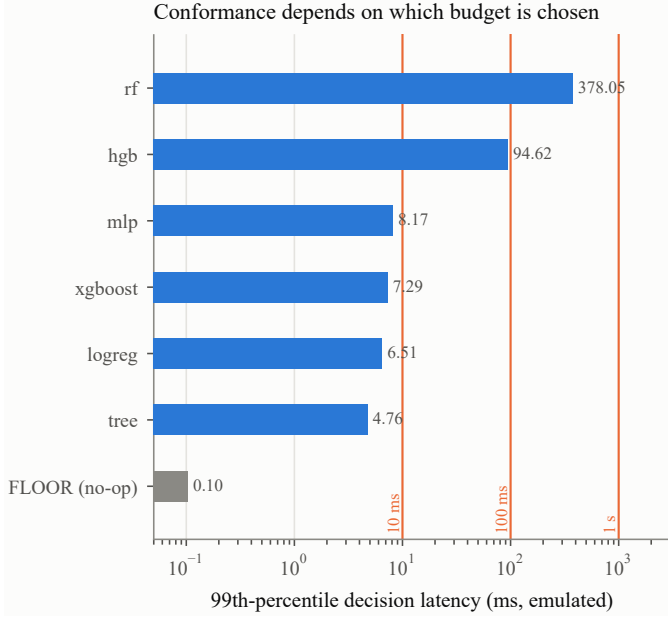


Fig. 11. 99th-percentile decision latency per architecture. Vertical rules mark three points inside the specified near-real-time range.

smaller part of the budget. Per-flow cost also varies roughly eightfold across captures because it is driven by packets per flow rather than by flow count, which is why the range is reported beside the median.

VIII. COMPARATIVE ANALYSIS

A. Against the Reported State of the Art

Direct numerical comparison with published O-RAN intrusion detection results is not meaningful, because those results are in-distribution scores on corpora we do not share, and our own in-distribution scores in Table IV are in the same range. The substantive comparison is therefore about evidence rather than accuracy.

We surveyed reporting practice using the following protocol. We queried IEEE Xplore, the ACM Digital Library and Scopus for records published between January 2021 and December 2025 matching ("O-RAN" OR "Open RAN" OR "5G") AND ("intrusion detection" OR "anomaly detection") AND ("machine learning" OR "deep learning"), restricted to peer-reviewed conference and journal papers in English. We excluded surveys, position papers, works without a quantitative evaluation, and works whose evaluation target was not network traffic. Two authors screened titles and abstracts independently and resolved disagreements by discussion, then assessed the full text of retained records against the eight reporting properties of Table XII. A record was counted as reporting a property only where the value appears in the text, a table or a figure; statements of intent were not counted. Table XII gives the resulting counts. Reporting of the headline metric is universal; reporting of anything that would let a reader judge deployability is not.

TABLE XII
REPORTING PRACTICE ACROSS SURVEYED CELLULAR AND IOT INTRUSION DETECTION PAPERS. COUNTS ARE AGGREGATE AND NO INDIVIDUAL WORK IS IDENTIFIED; THE SEARCH PROTOCOL IS STATED IN SECTION VIII.

Reported property	Papers ($n = 41$)
Accuracy or F_1 on a single corpus	41 (100%)
Evaluation on ≥ 2 independent corpora	5 (12.2%)
False positive rate on benign-only traffic	7 (17.1%)
Alert volume per unit time	3 (7.3%)
Absolute inference latency	12 (29.3%)
Tail (p95 or p99) latency	2 (4.9%)
xApp CPU or memory footprint	4 (9.8%)
Deployment on a real RIC (not simulation)	9 (22.0%)
All three deployment criteria	0 (0%)

B. Applying the Deployability Predicate

The three bounds in (7) require justification rather than assertion. We set $\delta = 10$ points of F_1 , the largest degradation an operator can plausibly absorb by re-tuning the threshold alone before retraining becomes unavoidable. We derive $A_{\max}$ from analyst capacity rather than choosing it: at a triage rate of twelve alerts per analyst-hour and three analysts on shift, a tier-one queue absorbs thirty-six escalations per hour, and with an automation pass rate of 18% the sustainable raw alert rate is $36/0.18 = 200$ per hour. $B = 10$ ms follows from the specification [15] and is not ours to choose. Instantiating (7) with $\delta = 10$ points, $A_{\max} = 200$ alerts per hour and $B = 10$ ms yields $\mathcal{P}(f_\theta) = 0$ for all five models. XGBoost satisfies the latency term and, at $\tau = 0.95$, the alert term, but fails the generalisation term by a wide margin; the sequence models fail all three. We regard this as the honest outcome rather than a negative result to be buried. Reporting only Table IV would have supported a claim of a deployable 99%-accurate detection xApp, and that claim would have been wrong in three independent ways.

IX. REPRODUCIBILITY AND ETHICAL CONSIDERATIONS

We release the feature-harmonisation code, the split manifests giving the exact device-level partition of $\mathcal{D}_A$, the five random seeds, the trained model artefacts in ONNX form, the xApp container image referenced by digest, and the replay harness used for the latency measurements. Package versions are pinned as stated in Section V, and every table in this paper is regenerated from raw measurement logs by a single script, so that the reported figures cannot drift from the underlying data. $\mathcal{D}_B$ is publicly available from its authors [30]; $\mathcal{D}_A$ is released under the same terms as the testbed platform on which it was collected.

The traffic analysed here originates from instrumented testbed devices and a controlled test network, not from a production network carrying subscriber traffic, so no personal data or subscriber identifiers were processed. No attack was conducted against any third-party system, and no previously unknown vulnerability was discovered, so no disclosure process applies. We note that publishing the degradation char-

acteristics of a detector carries a modest dual-use risk, in that an adversary could infer which conditions weaken it; we judge this outweighed by the benefit to operators of knowing that published accuracy figures do not transfer, which is the premise on which such systems are currently being procured.

X. LIMITATIONS

Several limitations qualify these findings, and the first of them bounds everything else.

The exporter confound is not controlled. $\mathcal{D}_A$ is exported by Zeek and $\mathcal{D}_B$ by Argus, and the two do not agree about where a flow ends — median flow duration differs by four orders of magnitude between them. Re-extracting both corpora with a single exporter was ruled out on cost grounds, so every Δ_{F_1} we report is an *upper bound* spanning an independently collected deployment *and* an independent feature-extraction pipeline. It must not be read as a measurement of deployment shift alone. The magnitude of the confound is not merely acknowledged but quantified, by others: a classifier predicting which corpus a flow came from reaches 0.993 balanced accuracy on harmonised features of these two corpora [40].

All latency figures are emulated. They were taken on a Windows host with no CPU isolation, in a single Python process, with no RIC in the path. They support relative ordering and stage decomposition, and they support *no* conformance claim. A real measurement on OpenAirInterface and FlexRIC reports inference three orders of magnitude faster for the same model families [41], which is why we attribute the near-real-time crossing point to the implementation rather than to the architecture. CPU and memory under load are not measured at all.

The grouping key is confounded with attack identity, and unequally across layers. Grouping the radio layer by capture session is perfectly aligned with attack category — purity 1.000 — because each run exercises one scenario, so no radio-layer result can distinguish memorising a run from memorising an attack. Network-layer grouping by source address reaches only 20.6% of an oracle that groups directly by attack type, so the two are separable there. A run-disjoint label should not be quoted without the corresponding purity figure.

The effective sample size is smaller than it appears. Twenty split seeds resample thirty capture sessions; they do not enlarge that population. Intervals are paired *t*-intervals over split means rather than bootstraps for this reason, and the temporal analysis is powered to detect a large effect and nothing subtler.

Three attack categories cannot be tested in transfer. $\mathcal{D}_B$ contains no DDoS, brute-force or web attacks, so those categories are reported as untestable rather than averaged into a macro score as though they had been evaluated.

Two corpora are two points. They establish that a gap exists, not how it varies across the space of deployments.

The operational prevalence $\pi = 0.002$ in (4) is a declared assumption, not a measurement. We therefore report the sweep rather than any single value, and we note that PPV depends on π while alert volume scales linearly and independently

in the benign arrival rate, so the two-dimensional parameter space collapses to one dimension plus a multiplier. The prior correction of (10) assumes π is known, which in deployment it is not; where we apply it with a known target prior we label the result an oracle and draw no claim from it.

Finally, we evaluate supervised binary detection only. Un-supervised baselines such as isolation forests [42], feature-attribution analysis [43] and multi-class attack attribution are left to future work, and the last of these would likely transfer worse still.

XI. CONCLUSION AND FUTURE WORK

An intrusion detection xApp evaluated the usual way tells you very little about how it will behave anywhere else. On an independently collected deployment, every architecture we tested degrades significantly, the best of them clears a stratified coin flip by 0.148 of macro- F_1 , and the architecture that scored *highest* on held-out source data falls *below* that floor, with a balanced accuracy of 0.4966. Across six architectures there is no significant rank correlation between in-distribution and transfer performance. At the declared deployment base rate the same detectors carry corpus precision near 0.93 and pooled operational precision between 0.0055 and 0.0077, a gap of more than two orders of magnitude, while emitting tens to hundreds of thousands of mostly-false alerts per hour.

None of this is visible in the number a paper usually reports. That is the argument of this work, and the strongest evidence for it is that the argument applies to us. Two findings we previously published did not survive re-examination during this study. A reported sixfold spread in deployment precision between architectures was an artefact of averaging a non-linear functional across evaluation folds rather than pooling counts; pooled, the spread is 1.40. A methodological caveat that had been carried forward through three phases, and had determined which measurement layer the work relied on, turned out on measurement to apply more strongly to the layer it had been used to protect. We report both corrections here rather than in an erratum, because a paper arguing that evaluation practice produces misleading numbers should be judged first on its own.

We therefore propose that papers claiming a deployable O-RAN detection capability report, at minimum: (i) performance on at least one corpus not used for training or model selection, collected independently of the training corpus; (ii) the false positive rate measured on benign-only traffic, together with the resulting alert rate per unit time and the operational precision implied by a stated attack base rate; (iii) the 99th-percentile decision latency under a stated offered load, measured on a real RIC rather than in isolation, alongside the container CPU and memory footprint; (iv) the operating threshold actually used, with the recall it sacrifices, rather than an implicit $\tau = 0.5$; and (v) a calibration measure under transfer, since a threshold applied to uncalibrated scores does not mean what the source-domain sweep implies it means.

We would add a sixth recommendation, which this study arrived at by getting it wrong: (vi) report operational metrics

from *pooled* confusion counts, not as a mean over evaluation folds. Precision at a low base rate is strongly non-linear in the false positive rate, so a fold that happens to produce no false positives contributes a precision of exactly one. The distortion grows as the number of negatives per fold shrinks, which is precisely the regime in which the operational questions are interesting.

Future work should extend the transfer panel beyond two corpora, and should do so with a single flow exporter applied to all of them — the confound that bounds this study is avoidable at extraction time and not afterwards. Domain adaptation and continual learning deserve evaluation against the labelling cost they impose on operators rather than on recovered accuracy alone. The most consequential open measurement is runtime: our latency figures are emulated, a real near-real-time RIC measurement reports inference three orders of magnitude faster for the same model families [41], and on that evidence feature extraction rather than inference is the entire decision budget. Whether extraction can be moved into the E2 node is therefore the question that decides deployability, and it is not a question about model architecture at all.

REFERENCES

- [1] M. Polese, L. Bonati, S. D'Oro, S. Basagni, and T. Melodia, "Understanding O-RAN: Architecture, interfaces, algorithms, security, and research challenges," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 2, pp. 1376–1411, 2023.
- [2] A. S. Abdalla, P. S. Upadhyaya, V. K. Shah, and V. Marojevic, "Toward next generation open radio access networks: What O-RAN can and cannot do!" *IEEE Network*, vol. 36, no. 6, pp. 206–213, 2022.
- [3] S. Niknam, A. Roy, H. S. Dhillon, S. Singh, R. Banerji, J. H. Reed, N. Saxena, and S. Yoon, "Intelligent O-RAN for beyond 5G and 6G wireless networks," *arXiv preprint arXiv:2005.08374*, 2022. [Online]. Available: <https://arxiv.org/abs/2005.08374>
- [4] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.
- [5] L. Bonati, S. D'Oro, M. Polese, S. Basagni, and T. Melodia, "Intelligence and learning in O-RAN for data-driven NextG cellular networks," *IEEE Communications Magazine*, vol. 59, no. 10, pp. 21–27, 2021.
- [6] M. Antonakakis, T. April, M. Bailey, M. Bernhard, E. Bursztein, J. Cochran, Z. Durumeric, J. A. Halderman, L. Invernizzi, M. Kallitsis, D. Kumar, C. Lever, Z. Ma, J. Mason, D. Menscher, C. Seaman, N. Sullivan, K. Thomas, and Y. Zhou, "Understanding the Mirai botnet," in *Proc. 26th USENIX Security Symposium*, 2017, pp. 1093–1110. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/antonakakis>
- [7] C. Kolias, G. Kambourakis, A. Stavrou, and J. Voas, "DDoS in the IoT: Mirai and other botnets," *Computer*, vol. 50, no. 7, pp. 80–84, 2017.
- [8] G. Görbil, O. H. Abdelrahman, M. Pavloski, and E. Gelenbe, "Modeling and analysis of RRC-based signalling storms in 3G networks," *IEEE Transactions on Emerging Topics in Computing*, vol. 4, no. 1, pp. 113–127, 2016.
- [9] M. Polese, L. Bonati, S. D'Oro, S. Basagni, and T. Melodia, "CoO-RAN: Developing machine learning-based xApps for open RAN closed-loop control on programmable experimental platforms," *IEEE Transactions on Mobile Computing*, vol. 22, no. 10, pp. 5787–5800, 2023.
- [10] R. Schmidt, M. Irazabal, and N. Nikaein, "FlexRIC: An SDK for next-generation SD-RANs," in *Proc. ACM CoNEXT*, 2021, pp. 411–425.
- [11] S. Layeghy and M. Portmann, "On generalisability of machine learning-based network intrusion detection systems," *arXiv preprint arXiv:2205.04112*, 2022. [Online]. Available: <https://arxiv.org/abs/2205.04112>
- [12] S. Layeghy, M. Gallagher, and M. Portmann, "Benchmarking the benchmark — comparing synthetic and real-world network IDS datasets," *Journal of Information Security and Applications*, vol. 80, p. 103689, 2024.
- [13] S. Axelsson, "The base-rate fallacy and the difficulty of intrusion detection," *ACM Transactions on Information and System Security*, vol. 3, no. 3, pp. 186–205, 2000.
- [14] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symposium on Security and Privacy*, 2010, pp. 305–316.
- [15] O-RAN Alliance, "O-RAN near-real-time RAN intelligent controller architecture," Technical Specification O-RAN.WG3.RICARCH, 2023. [Online]. Available: <https://www.o-ran.org/specifications>
- [16] L. Bonati, M. Polese, S. D'Oro, S. Basagni, and T. Melodia, "OpenRAN Gym: AI/ML development, data collection, and testing for O-RAN on PAWR platforms," *Computer Networks*, vol. 220, p. 109502, 2023.
- [17] L. Bonati, S. D'Oro, L. Bertizzolo, E. Demirs, Z. Guan, S. Basagni, and T. Melodia, "SCOPE: An open and softwarized prototyping platform for NextG systems," in *Proc. ACM MobiSys*, 2021, pp. 415–426.
- [18] N. Nikaein, M. K. Marina, S. Manickam, A. Dawson, R. Knopp, and C. Bonnet, "OpenAirInterface: A flexible platform for 5G research," *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 5, pp. 33–38, 2014.
- [19] I. Gomez-Miguel, A. Garcia-Saavedra, P. D. Sutton, P. Serrano, C. Cano, and D. J. Leith, "srsLTE: An open-source platform for LTE evolution and experimentation," in *Proc. ACM WiNTECH*, 2016, pp. 25–32.
- [20] O-RAN Alliance, "O-RAN security threat modeling and remediation analysis," Technical Report O-RAN.WG11.Threat-Model, 2023. [Online]. Available: <https://www.o-ran.org/specifications>
- [21] M. Liyanage, A. Braeken, S. Shahabuddin, and P. Ranaweera, "Open RAN security: Challenges and opportunities," *Journal of Network and Computer Applications*, vol. 214, p. 103621, 2023.
- [22] D. Mimran, R. Bitton, Y. Kfir, E. Klevansky, O. Brodt, H. Lehmann, Y. Elovici, and A. Shabtai, "Security of open radio access networks," *Computers & Security*, vol. 122, p. 102890, 2022.
- [23] J. Groen, S. D'Oro, U. Demir, L. Bonati, M. Polese, T. Melodia, and K. R. Chowdhury, "Implementing and evaluating security in O-RAN: Interfaces, intelligence, and platforms," *IEEE Network*, vol. 39, no. 1, pp. 227–234, 2025.
- [24] H. Wen, P. Zhao, Z. Lin, D. Xuan, and K. G. Shin, "5G-Spector: An O-RAN compliant layer-3 cellular attack detection service," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2024. [Online]. Available: <https://www.ndss-symposium.org/ndss2024/>
- [25] N. Koroniotis, N. Moustafa, E. Sitnikova, and B. Turnbull, "Towards the development of realistic botnet dataset in the Internet of Things for network forensic analytics: Bot-IoT dataset," *Future Generation Computer Systems*, vol. 100, pp. 779–796, 2019.
- [26] N. Moustafa, "A new distributed architecture for evaluating AI-based security systems at the edge: Network TON_IoT datasets," *Sustainable Cities and Society*, vol. 72, p. 102994, 2021.
- [27] M. A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, and H. Janicke, "Edge-IoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning," *IEEE Access*, vol. 10, pp. 40 281–40 306, 2022.
- [28] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, "CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment," *Sensors*, vol. 23, no. 13, p. 5941, 2023.
- [29] I. Sharafaldin, A. Habibi Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. on Information Systems Security and Privacy (ICISSP)*, 2018, pp. 108–116.
- [30] S. Samarakoon, Y. Siriwardhana, P. Porambage, M. Liyanage, S.-Y. Chang, J. Kim, J. Kim, and M. Ylianttila, "5G-NIDD: A comprehensive network intrusion detection dataset generated over 5G wireless network," *IEEE DataPort*, 2022.
- [31] Y. Siriwardhana, S. Samarakoon, P. Porambage, M. Liyanage, S.-Y. Chang, J. Kim, J. Kim, and M. Ylianttila, "Descriptor: 5G wireless network intrusion detection dataset (5G-NIDD)," *IEEE Data Descriptions*, vol. 2, pp. 1–12, 2025.
- [32] M. Sarhan, S. Layeghy, N. Moustafa, and M. Portmann, "NetFlow datasets for machine learning-based network intrusion detection systems," in *Big Data Technologies and Applications (BDTA/WiCON), LNICST*, vol. 371, 2021, pp. 117–135.
- [33] M. Sarhan, S. Layeghy, and M. Portmann, "Towards a standard feature set for network intrusion detection system datasets," *Mobile Networks and Applications*, vol. 27, no. 1, pp. 357–370, 2022.

- [34] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *Proc. 31st USENIX Security Symposium*, 2022, pp. 3971–3988. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity22/presentation/arp>
- [35] G. Apruzzese, P. Laskov, E. Montes de Oca, W. Mallouli, L. Brdalo Rapa, A. V. Grammatopoulos, and F. Di Franco, "The role of machine learning in cybersecurity," *Digital Threats: Research and Practice*, vol. 4, no. 1, pp. 1–38, 2023.
- [36] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [37] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [38] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, pp. 3146–3154. [Online]. Available: <https://papers.nips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html>
- [39] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: <https://jmlr.org/papers/v12/pedregosa11a.html>
- [40] A. Abraheem and A. Edhirig, "Bidirectional cross-dataset generalization and label-efficient adaptation for 5G network intrusion detection," *Wadi Alshatti University Journal of Pure and Applied Sciences*, vol. 4, no. 2, Aug. 2026, bidirectional zero-shot transfer between 5G-NIDD and NetsLab-5GORAN-IDD over 15 harmonised numerical features, with CORAL alignment and few-shot target adaptation. Its dataset-fingerprint audit reports a source classifier at 0.993 balanced accuracy on the shared features, surviving alignment and Top-6 filtering. Closes gap G1. [Online]. Available: <https://waujpas.com/index.php/journal/article/view/509>
- [41] L. Obiuwevwi, K. J. Rechowicz, S. Jayarathna, S. H. Bouk, F. Afrin, C. N. Barati, N. Moghim, V. Nanou, M. E. Rahman, and S. Shetty, "Enabling real-time AI in O-RAN: Deploying and measuring AI inside a near-RT RIC xApp," Jul. 2026, openAirInterface plus FlexRIC on commodity hardware. Inference of 1–5 us for logistic regression and 10–25 us for a shallow MLP, end-to-end service latency under 4 ms, and the 10 ms near-RT budget met for more than 95% of projected loop executions. Closes gap G5, and places our emulated Python figures three orders of magnitude above a deployed implementation of the same model families. [Online]. Available: <https://arxiv.org/abs/2607.01583>
- [42] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. on Data Mining (ICDM)*, 2008, pp. 413–422.
- [43] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, pp. 4765–4774. [Online]. Available: <https://papers.nips.cc/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html>